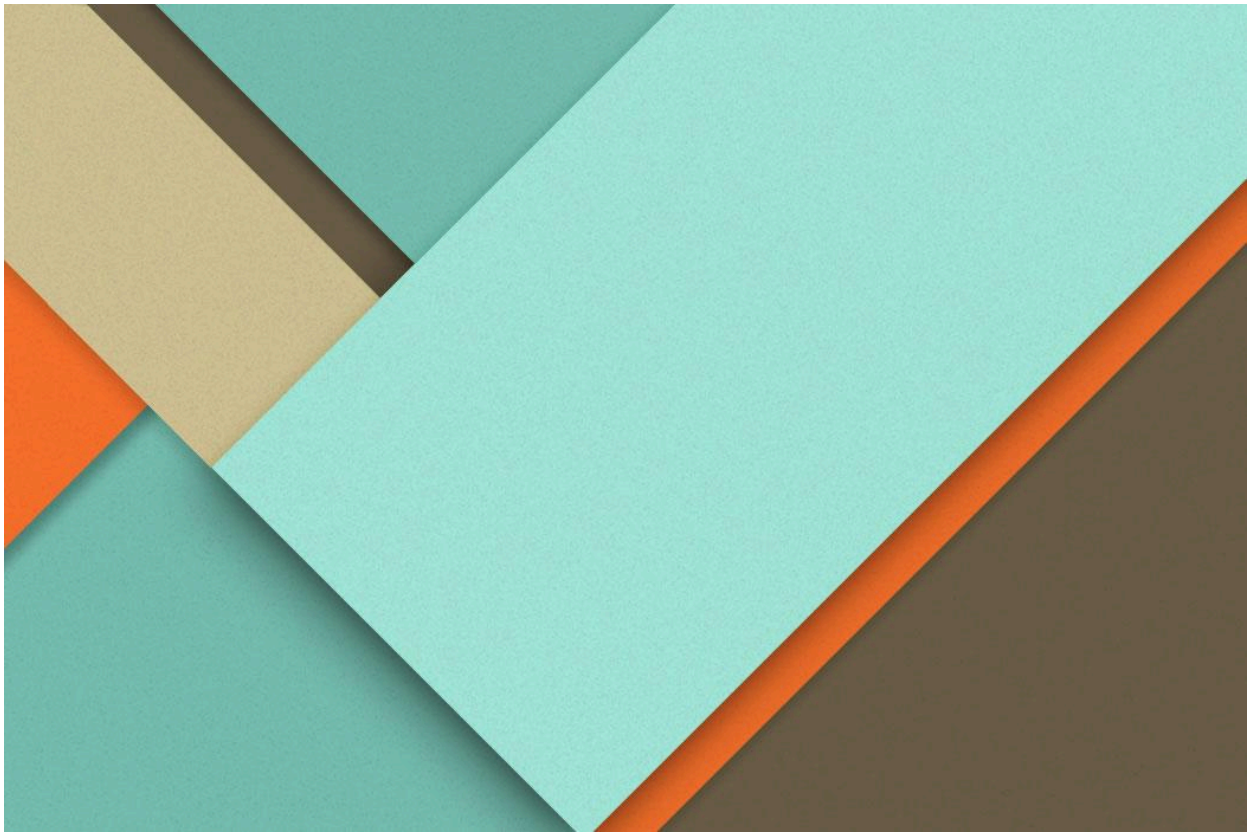


CS7504

Assessment 1:

Course: CS7504 – Security for DevOps

Student Name: Louie Poland

Student ID: 22301774

Executive Summary & Introduction

The following document is about an automated, secure multi-tier web application architecture deployed on Microsoft Azure.

The application is here to transition an internal system to a more modern cloud system that includes:

1. Linux Web Server: Ubuntu 22.04 LTS hosting Nginx and PHP runtime, exposed via an Azure Public IP over HTTP (TCP 80) and SSH (TCP 22).
2. Linux Database Server: Ubuntu 22.04 LTS is hosting MySQL Server, isolated within a private virtual network subnet and has no direct public access.
3. Azure Key Vault: A dedicated, hardware-backed key vault (kv-cs7504-louie) storing sensitive credentials, enforcing a least-privilege security policies.

This documentation details the Infrastructure as Code (IaC) implementation via Terraform, Configuration Management via Ansible, Secure Secret Injection via Azure Key Vault, dynamic web application retrieval, critical theoretical evaluations, and a structured Source Code Management (SCM) workflow executed in GitHub.

Part 1: Infrastructure, Automation, and DevOps Architecture (LO1)

Task 1: Terraform Infrastructure Provisioning

Architecture Overviewa

The cloud infrastructure is made using HashiCorp Terraform using modular config files. The network architecture isolates the Database Virtual Machine from any public internet while enabling secure internal communication from the Web Server.

Azure Resources

- Resource Group: rg-cs7504-assignment1 (Location: japaneast)
- Virtual Network (VNet): vnet-cs7504 (10.0.0.0/16)
- Subnet: subnet-internal (10.0.1.0/24)
- Network Security Group (NSG): nsg-cs7504 with ingress rules for SSH (22) and HTTP (80).
- Web VM: vm-webserver (Standard_B1s, Ubuntu 22.04 LTS, attached to Dynamic Public IP pip-webserver).
- DB VM: vm-dbserver (Standard_B1s, Ubuntu 22.04 LTS, Private IP only).
- Key Vault: kv-cs7504-louie containing the secret database-password.

Task 2: Ansible Server Configuration

Automation Strategy & Idempotency

Ansible playbooks are allowed to configure software packages, create database, and set permissions.

Task 3: Web Application Deployment & Database Retrieval

Application Code

The web interface connects dynamically to the MySQL database on the database virtual machine (10.0.1.5) via PDO over port 3306.

Task 4: Secure Secret Management using Azure Key Vault

Security Compliance Strategy

Preventing hardcoded passwords within repository or deployment:

1. Database secrets are declared as variables in Terraform and injected directly into Azure Key Vault (kv-cs7504-louie).
2. Deployment workflows access the secret using Azure CLI or Managed Identity injection (az keyvault secret show --name database-password --vault-name kv-cs7504-louie).
3. Source files (.tf, .yml, .php, .ini) checked into Git utilize environment variable placeholders (getenv('DB_PASSWORD')), keeping plain-text secrets completely out of version control.

Report 1: Critical Evaluation of Terraform State Management

Evaluation of Terraform State Management

Terraform state is the core piece of Infrastructure as Code that connects your real-world cloud resources (like Azure virtual machines) to your local .tf code files. Instead of constantly hitting cloud APIs to figure out what's running, Terraform uses a metadata file called terraform.tfstate. This file keeps track of your resource IDs, settings, versions, and dependencies.

Keeping state managed properly ensures your infrastructure stays consistent. Whenever you run terraform plan or terraform apply, Terraform compares your local code against the state file and the actual cloud setup to see what needs to change. This stops you from accidentally creating duplicate resources or deleting things by mistake as it runs it through its own safety cgeck.. In team environments, it's best practice to store state remotely

However state management does come with some challenges and security risks. The biggest issue is sensitive data handling. For example Terraform state files store sensitive values (like database passwords) in plain text unless you manually set up explicit encryption. Another common problem is configuration drift: if someone makes a manual change in the Azure Portal outside of Terraform, the file gets out of sync. Fixing this requires running manual cleanup commands.

State management is essential for automated CI/CD pipelines and team collaboration. Stored in a secure remote backend with strict access controls keeps your infrastructure reliable and gives your team a single source of truth.

Report 2: Secret Management Comparison – HashiCorp Vault vs. Ansible Vault

Managing credentials like database passwords, API keys, and SSH keys properly is crucial in DevOps so things don't get accidentally leaked. HashiCorp Vault and Ansible Vault are two common ways to handle this, but they work quite differently in practice.

	HashiCorp Vault	Ansible Vault
Security	A central service with defined permissions, dynamic secret generation and high availability.	File-level encryption (AES-256) built directly into Ansible playbooks/variables.
Scalability	Handles app secrets across entire platforms, multi-cloud setups, and microservices.	Built specifically for encrypting Ansible tasks, variables, and playbooks.
Integration	Connects via REST APIs, Kubernetes sidecars and Terraform providers.	Works directly inside the Ansible command line
setup	Harder to set up; requires managing a dedicated server.	Super easy to use; requires no extra servers or setup.

Recommendation

- Ansible Vault is ideal for small projects where you just need a quick, simple way to encrypt secrets.
- HashiCorp Vault is better for larger cloud setups and automated CI/CD pipelines that need strict access controls, automatic password rotation, and any temporary credentials

References

- Azure Documentation. (2026). Azure Key Vault security baseline. Microsoft Docs. https://learn.microsoft.com/en-us/azure/key-vault/general/security-features
- HashiCorp. (2026). Terraform state management and remote backends. Terraform Documentation. https://developer.hashicorp.com/terraform/language/state
- Red Hat Ansible. (2026). Ansible Vault user guide. Ansible Documentation. https://docs.ansible.com/ansible/latest/vault_guide/index.html
- Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.